



스프린트 스크럼

1주차 스크럼: 7월 8일 (화)

- 기간: 7월 8일 ~ 7월 13일
- 주요 목표:
 - AI 모델 PoC 테스트: YOLOv11, MiDaS, RoomNet 테스트 및 결과 검토
 - API 기본 설계: `/api/layout/detect?height_cm=` 설계 및 입력 구조 정의
 - 프론트엔드 UI 기획: 사진 업로드, 층고 입력 필드, 캔버스 영역 UI 설계
- 논의 사항:
 - 모델 경량화 여부 (YOLOv11n vs YOLOv11s)
 - 층고 미입력 시 기본값 적용 방식
 - 각 모델의 종속성, API 통합 방식 검토

2주차 스크럼: 7월 15일 (화)

- 기간: 7월 14일 ~ 7월 20일
- 주요 목표:
 - API 더미 구현 완료 및 샘플 이미지 테스트 통합

- 이미지 업로드 UI 개발, 층고 입력 필드 구현
 - 캔버스 프레임워크 선택 (Konva.js 등) 및 기본 시각화 구성
 - 논의 사항:
 - 업로드 이미지의 해상도 제한 및 변환 방식
 - 프론트 → 백엔드 전송 포맷 (multipart/form-data)
 - YOLOv11 결과 좌표 정규화 방식
-

3주차 스크럼: 7월 22일 (화)

- 기간: 7월 21일 ~ 7월 27일
 - 주요 목표:
 - YOLOv11, MiDaS, RoomNet 실제 모델 연동
 - 가구 좌표 + 깊이 정보 + 외곽 추정 결과 종합 구조 설계
 - 프론트엔드 평면도 2D 캔버스 시각화 구현
 - 논의 사항:
 - RoomNet 실패 시 fallback 처리 여부 (YOLO + OpenCV)
 - 이미지 왜곡 보정 알고리즘 효과 확인
 - 캔버스 단위 환산 로직: 픽셀 ↔ cm 변환 기준
-

4주차 스크럼: 7월 29일 (화)

- 기간: 7월 28일 ~ 8월 3일
 - 주요 목표:
 - 전체 애플리케이션 Docker화 (FastAPI + Front)
 - 로컬 테스트 환경에서 API 호출 및 결과 확인
 - 모델 추론 속도 개선 및 캐시 전략 도입 검토
 - 논의 사항:
 - 이미지 크기 제한 및 메모리 관리 전략
 - 모델 로딩 시점 (서버 시작 vs 요청 시 Lazy Load)
 - Docker 환경에서 OpenCV/torch 종속성 최적화
-

5주차 스크럼: 8월 5일 (화)

- 기간: 8월 4일 ~ 8월 10일
 - 주요 목표:
 - Kubernetes 클러스터에 앱 배포 시작
 - YAML 구성 (Deployment, Service) 작성 및 테스트
 - 이미지 업로드 + 분석 + 시각화 흐름 완전 통합 테스트
 - 논의 사항:
 - Pod 간 통신 설정 및 LoadBalancer/IP 접근 테스트
 - Ingress Controller 구성 및 라우팅 방식 검토
 - 도커 이미지 빌드 시 모델 포함 여부 (용량 고려)
-

6주차 스크럼: 8월 12일 (화)

- 기간: 8월 11일 ~ 8월 17일
 - 주요 목표:
 - GitHub Actions 기반 CI/CD 파이프라인 구축
 - Ingress 경로 정리 (/api/layout , /static) 및 보안 인증 설정
 - 모델 및 API 상태 모니터링 기능 적용 (Prometheus 등)
 - 논의 사항:
 - S3, 모델 서버, 분석 API 분리 여부
 - Github Secrets에 모델 토큰, S3 Key 등 통합 관리
 - 실시간 배포 후 오류 복구 플랜 (Rollback 방식)
-

7주차 스크럼: 8월 19일 (화)

- 기간: 8월 18일 ~ 8월 20일
- 주요 목표:
 - 전체 기능 통합 테스트 (업로드 → 분석 → 평면도 출력)
 - 운영 문서화 및 README 정리, API 명세서 작성
 - 시연/발표용 자료 구성 및 실제 테스트 영상 녹화

- 논의 사항:

- fallback 메시지, 에러 시나리오 대응 안내 메시지 통합 여부
- Canvas 화면 캡처 or 다운로드 기능 포함 여부
- 발표 시 강조할 핵심 가치 및 기술적 차별화 포인트

스프린트 스크럼 요약 (1~7주차) - [2D 평면도 생성 기능]

 **기능명:** 방 사진 기반 객체 감지 및 층고·깊이 기반 공간 추정 기능

 목적:

사용자가 업로드한 방 사진을 기반으로 AI가 가구를 감지하고, 층고 및 공간 왜곡 보정 기술 (깊이 추정, 레이아웃 추정 등)을 활용해 방의 실제 크기와 구조를 자동으로 2D 평면도로 시각화한다.

연동 EPIC

- EPIC-1: 핵심 AI 서비스 개발 및 API 구축
- EPIC-2: 웹 애플리케이션 프론트엔드/백엔드 개발
- EPIC-3: CI/CD 파이프라인 구축 및 도커/쿠버네티스 배포 자동화

1주차 (7/8~7/13)

날짜	할 일	담당	상태
7/8 (월)	YOLOv11, MiDaS, RoomNet 모델 PoC 테스트	AI	
7/9 (화)	<code>/api/layout/detect</code> API 설계 시작	백엔드	
7/10 (수)	업로드 UI 기획 및 캔버스 프레임워크 검토	프론트	
7/11 (목)	층고 기본값 처리 및 모델 연동 기획	백엔드	
7/12 (금)	기능 구성안 기반 와이어프레임 작성	프론트	

2주차 (7/14~7/20)

날짜	할 일	담당	상태
7/15 (월)	업로드 UI 완성 및 캔버스 배치	프론트	
7/16 (화)	총고 입력 필드 완성, 유효성 검사 추가	프론트	
7/17 (수)	모델 좌표 정규화 및 전처리 테스트	AI/백엔드	
7/18 (목)	업로드된 이미지 해상도 자동 조정 기능 구현	프론트/백	
7/19 (금)	Front → Back 연동 테스트 및 캔버스 렌더링 확인	전체	

3주차 (7/21~7/27)

날짜	할 일	담당	상태
7/22 (월)	평면도 캔버스 구성 및 좌표 → cm 환산 공식 적용	프론트	
7/23 (화)	fallback 로직: RoomNet 실패 시 YOLO+OpenCV 대응 구현	백엔드	
7/24 (수)	전체 시각화 플로우 점검 및 오류 시 메시지 처리	프론트/백	
7/25 (목)	테스트 이미지 기반 렌더링 결과 품질 검토	전체	
7/26 (금)	배포 전 통합 테스트 및 모델 추론시간 측정	전체	

4주차 (7/28~8/3)

날짜	할 일	담당	상태
7/29 (월)	OpenCV/torch 의존성 포함 Dockerfile 작성	백엔드	
7/30 (화)	로컬 테스트용 docker-compose 구성	전체	
7/31 (수)	모델 Lazy Load vs Static Load 테스트 비교	AI	
8/1 (목)	리소스 사용량 측정 및 캐시 전략 적용 검토	전체	

날짜	할 일	담당	상태
8/2 (금)	로컬 환경 통합 테스트 완료	전체	

5주차 (8/4~8/10)

날짜	할 일	담당	상태
8/5 (월)	Deployment, Service YAML 작성	DevOps	
8/6 (화)	Ingress Controller 설정 및 도메인 연동	DevOps	
8/7 (수)	앱 K8S 배포 및 외부 노출 테스트	DevOps	
8/8 (목)	백엔드 연동 확인 및 실 서비스 점검	백엔드	
8/9 (금)	테스트 자동화 스크립트 적용	전체	

6주차 (8/11~8/17)

날짜	할 일	담당	상태
8/12 (월)	GitHub Actions 배포 연동 시작	DevOps	
8/13 (화)	모델 및 API 상태 모니터링 도구 적용	DevOps	
8/14 (수)	Ingress 보안 설정 적용 (TLS 등)	DevOps	
8/15 (목)	배포된 앱 성능 테스트 및 실시간 분석	전체	
8/16 (금)	에러 복구 전략 및 Rollback 테스트	전체	

7주차 (8/18~8/20)

날짜	할 일	담당	상태
8/18 (월)	통합 시연 리허설 및 체크리스트 작성	전체	
8/19 (화)	발표용 PPT, 소개자료, API 명세 문서 작성	전체	
8/20 (수)	최종 배포	전체	